Adım 1.1 — Saat Ağacı: HSE → PLL → 168 MHz

Konu anlatımı ve soru-cevap · Güvenli Telemetri Düğümü (STM32F407 + FreeRTOS + lwIP + AEAD)

1. Hedef

Kart açılışta 16 MHz'lik dahili saatle (HSI) çalışır. STM32F407 en çok 168 MHz'e çıkabilir. Bu adımda kartın üzerindeki 8 MHz kristali (HSE) kullanıp bir PLL ile 168 MHz'lik sistem saati elde ettik. Sonraki adımlarda (UART, SPI, kriptografi) hız gerektiği için bu zemin lazım.

Dürüstlük notu: 168 MHz'e geçişi henüz **sayıyla ölçmedik**; bunu 1.2'de DWT cycle counter ile ölçüp docs/olcumler.md'ye yazacağız. Şimdilik kanıtımız: kod kilitlenmeden çalışıyor ve LED/buton eskisi gibi çalışıyor.

2. Temel kavramlar (sıfırdan)

Register, bit ve alan

Register, işlemcinin içindeki 32 bitlik bir hücredir. Her bit bir şalter gibi: 0 kapalı, 1 açık. Bit numaraları sağdan sola 0'dan 31'e gider. Bazı ayarlar birkaç bit yan yana durur; bu gruba **alan** (field) denir. Örnek: PPRE1 alanı CFGR'nin 12:10. bitleridir.

İsimler ne demek?

İsim	Açılımı	Ne iş yapar?
RCC	Reset and Clock Control	Çipin saat ve reset yöneticisi.
RCC_CR	Clock Control Register	Osilatörleri (HSE, PLL) açar/kapatır ve 'hazır' durumunu gösterir.
RCC_CFGR	Clock Configuration Register	Sistem saati kaynağı ve bölücüler (AHB/APB) burada.
RCC_PLLCFGR	PLL Configuration Register	PLL'in M, N, P, Q ve kaynak ayarları.
FLASH_ACR	Flash Access Control Register	Flash'tan okuma davranışı; içindeki LATENCY alanı wait state sayısıdır.
HSI	High Speed Internal	Dahili 16 MHz RC osilatör. Reset sonrası varsayılan. Kaba.
HSE	High Speed External	Kartın üzerindeki 8 MHz kristal. Daha hassas.
PLL	Phase-Locked Loop	Frekansı çarpan devre.

PLL: 8 MHz'i 168 MHz'e nasıl çıkardık?

```
HSE 8 MHz --/M--> 1 MHz --xN--> 336 MHz (VCO) --/P--> 168 MHz (sistem saati)
                                       \--/Q--> 48 MHz (USB için)

M = 8, N = 336, P = 2, Q = 7
```

- **M:** PLL girişini böler. Girişin 1–2 MHz aralığında olması gerekir: 8 / 8 = 1 MHz.
- **N:** Çarpar: 1 × 336 = 336 MHz. Bu ara değere VCO frekansı denir; sınırı 100–432 MHz.
- **P:** Sistem saatini üretir: 336 / 2 = 168 MHz.
- **Q:** USB gibi birimlerin 48 MHz saati: 336 / 7 = 48 MHz. USB kullanmıyoruz, ST'nin standart ayarını izledik.
- Formüller RM0090 6.3.2'de yazılı: f(VCO) = f(giriş) × N/M, f(sistem) = f(VCO)/P, f(USB) = f(VCO)/Q.

Yollar (bus) ve ön bölücüler

Çip içinde birimler farklı hızdaki yollara bağlıdır: **AHB** hızlı ana yoldur (işlemci, Flash, DMA, GPIO). **APB1** ve **APB2** daha yavaş yan yollardır (UART, SPI, zamanlayıcılar). Her yolun hızı, sistem saatinin bir **bölücüden** (prescaler) geçirilmesiyle elde edilir. APB1 en çok 42 MHz, APB2 en çok 84 MHz olabilir.

Bölücü	Açılımı	Bizim değer	Sonuç
HPRE	AHB PREscaler	÷1 (reset değeri)	168 MHz
PPRE1	APB1 PREscaler (CFGR 12:10)	÷4 → alan değeri 101	42 MHz
PPRE2	APB2 PREscaler (CFGR 15:13)	÷2 → alan değeri 100	84 MHz

PPRE alan değerleri: `0xx = ÷1, 100 = ÷2, 101 = ÷4, 110 = ÷8, 111 = ÷16`. Kodda `(5 << 10)`: 5 = ikilikte 101; alan 10. bitten başladığı için 10 bit sola kaydırılır.

Flash wait state (bekleme durumu)

Flash bellek yavaştır. İşlemci hızlandıkça Flash'tan okuma için ek saat çevrimi beklemek gerekir; bunlara **wait state** denir. Benzetme: kütüphaneci kitabı 30 saniyede getiriyor, sen daha hızlı soruyorsan kitap gelene kadar beklemek zorundasın.

- RM0090 tablosunda **"5 WS (6 CPU cycles)"** yazar: 5 ek bekleme çevrimi + okumanın kendi 1 çevrimi = toplam 6 çevrim. Register'a yazılan sayı bekleme sayısıdır, yani **5**: `FLASH_ACR = 5;`
- Tabloda **voltaj sütunları** var, çünkü düşük gerilimde transistörler yavaşlar ve Flash'ın cevap süresi uzar; aynı hız için daha çok bekleme gerekir.
- Gerilimi biz belirlemiyoruz: Discovery kartında USB'den gelen 5 V kartın üzerindeki regülatörle yaklaşık 3 V'a düşürülüp çipin VDD pinine verilir. Bu yüzden 2.7–3.6 V sütunu geçerli. (Teyit için kart kılavuzu UM1472.)
- 168 MHz tablo satırı: 150 < HCLK ≤ 168 MHz → 5 WS.

SW ve SWS

Sistem saatinin kaynağı CFGR'deki iki alanla yönetilir: **SW** (System clock Switch, bit 1:0) yazdığımız istektir: `00 = HSI, 01 = HSE, 10 = PLL`. **SWS** (System clock Switch Status, bit 3:2) donanımın yazdığı, salt okunur durumdur: "şu an gerçekten hangi saati kullanıyorum". SW'ye yazıp geçişin bittiğini SWS ile doğrularız.

3. Register adresleri

Register	Adres	Taban + ofset
RCC_CR	0x40023800	RCC tabanı + 0x00
RCC_PLLCFGR	0x40023804	RCC tabanı + 0x04
RCC_CFGR	0x40023808	RCC tabanı + 0x08
RCC_AHB1ENR	0x40023830	RCC tabanı + 0x30
FLASH_ACR	0x40023C00	Flash arayüzü tabanı + 0x00

Bu adresleri biz belirlemedik: ST, çipin bellek haritasını tasarlarken belirler ve donanıma gömer (RM0090, Table 1 *Register boundary addresses*). Kodda yazdığımız #define isimleri sadece bizim koyduğumuz etiketlerdir.

4. Kod (main.c, son hâli)

```
#define RCC_CR        (*(volatile uint32_t *)0x40023800)
#define RCC_PLLCFGR   (*(volatile uint32_t *)0x40023804)
#define RCC_CFGR      (*(volatile uint32_t *)0x40023808)
#define FLASH_ACR     (*(volatile uint32_t *)0x40023C00)

int main(void)
{
    RCC_CR |= (1 << 16);                // HSEON: kristali çalıştır
    while (!(RCC_CR & (1 << 17)));      // HSERDY: hazır olana kadar bekle

    FLASH_ACR = 5;                      // 168 MHz için 5 wait state
    RCC_CFGR |= (5 << 10);              // PPRE1 = /4 -> APB1 = 42 MHz
    RCC_CFGR |= (4 << 13);              // PPRE2 = /2 -> APB2 = 84 MHz

    RCC_PLLCFGR = (RCC_PLLCFGR & 0xF0000000) | (8 << 0) | (336 << 6)
                | (0 << 16) | (1 << 22) | (7 << 24);   // M=8 N=336 P=2 HSE Q=7
    RCC_CR |= (1 << 24);                // PLLON
    while (!(RCC_CR & (1 << 25)));      // PLLRDY

    RCC_CFGR |= (2 << 0);                           // SW = 10: PLL
    while ((RCC_CFGR & (3 << 2)) != (2 << 2));      // SWS = 10 olana kadar bekle
    ...                                             // (LED/buton kodu devam eder)
}
```

Sıra neden önemli?

- 1) HSE'yi aç ve hazır olmasını bekle. 2) Flash wait state'i ve bölücüleri **hızlanmadan önce** ayarla. 3) PLL'i yapılandır, aç, kilitlenmesini bekle. 4) En son sistem saatini PLL'e geç ve SWS ile doğrula.
- Wait state ayarlanmadan hızlanırsak işlemci Flash'tan yanlış okur; bölücüler ayarlanmadan APB1 42 MHz sınırını aşar. Kart kilitlenebilir.
- Tüm bunlar tek oturumda yapıldığı için PLL'e geçiş son satırda, en riskli an olarak yapılır.

Satır notları

- `|=` yalnızca ilgili biti 1 yapar, diğer bitlere dokunmaz.
- `FLASH_ACR = 5` ve `RCC_PLLCFGR = ...` satırlarında = kullandık çünkü tüm alanları kendi değerlerimizle yazıyoruz (ACR'nin diğer bitleri şimdilik 0 kalıyor).
- `while (...)` döngüleri "hazır" bitini bekler; hazır olmadan kullanmak saati bozar.
- `(3 << 2)`: iki bitlik bir **maske**; `RCC_CFGR & maske` yalnızca SWS alanını süzer.

5. Yaptığımız düzeltme: Reserved (ayrılmış) bitler

RM0090 RCC_PLLCFGR sayfasında: **"Bit 31:28 Reserved, must be kept at reset value."** yani ayrılmış bitlere dokunma, reset değerinde bırak. Reset değeri `0x2400 3010`; bit 29 reset'te 1. İlk verdiğim `RCC_PLLCFGR = ...` satırı tüm register'ı sıfırdan yazdığı için bu biti sıfırladı. Düzeltme:

```
RCC_PLLCFGR = (RCC_PLLCFGR & 0xF0000000) | ...;
```

Buna **oku-değiştir-yaz** (read-modify-write) denir: eski değerin korunacak bitlerini tut, kendi alanlarını ekle. Dersi: bir register'a tamamen yazmadan önce RM0090'da reserved bitleri ve reset değerini kontrol et.

6. Debug günlüğü: takılma

Kodu PLL'e geçiş satırıyla birlikte çalıştırınca LED yanmadı. Debugger, HSE bekleme döngüsünde (`while (!(RCC_CR & (1 << 17)))`) durdu. Aynı döngü bir önceki oturumda sorunsuz geçmişti. Kartın USB kablosunu çekip tekrar takınca program çalıştı.

- **Yöntem:** Suspend ile programı durdurup okun hangi satırda olduğuna bakmak, takılmayı en kısa yoldan lokalize eder.
- **Olası neden (doğrulanmadı):** Önceki oturumda kart PLL ile 168 MHz'de çalışıyordu; bir sonraki debug oturumundaki reset, saat durumunu tam temizlememiş olabilir. Kesin nedeni ölçmedik, bu yüzden bunu bir tahmin olarak not ediyorum.
- **Pratik kural:** Saat ayarıyla oynarken beklenmedik bir takılma görürsen kartın gücünü kesip tekrar dene.

7. Soru - cevap

Soru 1: Tablodaki "5 WS (6 CPU cycles)" ne demek?

5 ek bekleme çevrimi + okumanın kendi 1 çevrimi = 6 çevrim. Register'a yazılan LATENCY değeri bekleme sayısıdır: **5**.

Soru 2: Tablodaki voltaj değerleri neye göre belirleniyor?

Çipin besleme gerilimine (VDD) göre. Düşük gerilimde Flash daha yavaş cevap verir, daha çok bekleme gerekir. Discovery kartında VDD, USB'den gelen 5 V'tan kart üstündeki regülatörle yaklaşık 3 V'a düşürülür; biz değiştirmiyoruz.

Soru 3: Taban adresleri kim veriyor?

ST. Çipin bellek haritası tasarım aşamasında belirlenir ve donanıma gömülüdür. RM0090 Table 1'de yazar. Biz sadece bunlara isim veririz.

Soru 4: PLL ayarlarını tek satırda yazarken neden = kullandık da |= kullanmadık?

Register'ın reset değeri sıfır değil (varsayılan M, N, Q içeriyor). |= ile yazarsak eski bitler kalır ve yeni değerlerle karışır. = ile tüm alanları baştan yazıyoruz. Reserved bitleri korumak için eski değerin sadece o bitlerini maskeleyip ekledik.

8. Doğrulama ve yapılacaklar

- SFRs görünümünde `RCC → CFGR` okunurken beklenen değer `0x0000940A`: PPRE1 101 → 0x1400, PPRE2 100 → 0x8000, SW = 10 ve SWS = 10 → 0xA. **(Bu değeri henüz kontrol etmedik.)**
- 168 MHz'in gerçekten ölçülmesi: 1.2'de DWT cycle counter ile, sonuç docs/olcumler.md'ye yazılacak.
- FPU uyarısı hâlâ duruyor; FPU'yu ilerleyen adımlarda açacağız.

9. Kendini test et

- PLL zincirinde M, N, P, Q ne yapar? 8 MHz'ten 168 MHz'i hesapla.
- APB1 neden ÷4, APB2 neden ÷2?
- Wait state nedir? Ayarlamadan hızlanırsak ne olur?
- SW ile SWS arasındaki fark nedir? Neden ikisi de gerekli?
- PLL'e geçiş sırası neden bu şekilde (HSE → wait state/bölücü → PLL → SW)?